40 Days of LeetCode Challenge

40 days, 40 problems
DAY :2
LEETCODE PROBLEM NO. 1281 : Subtract the Product and Sum of Digits of an Integer
Problem Link : https://leetcode.com/problems/subtract-the-product-and-sum-of-digits-of-an-integer/description/
Easy · Bit Manipulation / Simulation
STEP BY STEP SOLUTION:-

Step 1. Understand the question :-

We are given a positive integer n, return the difference between the product of its digits and the sum of its digits. That is we need to:

1. Find the product of its digits — multiply all the digits together.
2. Find the sum of its digits — add all the digits together.
3. Subtract the sum from the product.

The final result is:

Answer = Product of digits - Sum of digits

Step 2. Translating it into simpler language with examples :-

Imagine that every digit in the number is a separate value.

Example 1:
Input: n = 234
Output: 15
Explanation:
Product of digits = 2 * 3 * 4 = 24
Sum of digits = 2 + 3 + 4 = 9
Result = 24 - 9 = 15

For the number 234, we have the digits 2, 3, and 4.

1. First, multiply them together. Then, add them together.
2. Finally, subtract the addition result from the multiplication result.

	n = 234	
2 digit1	3 digit2	4 digit3
Multiply the digits		2 x 3x 4 = 24
Add the digits		2 + 3 + 4 = 9
FINAL RESULT		24 - 9 = 15

Example 2:
Input: n = 4421
Output: 21
Explanation:
Product of digits = 4 * 4 * 2 * 1 = 32
Sum of digits = 4 + 4 + 2 + 1 = 11
Result = 32 - 11 = 21

For the number 4421, we have the digits 4, 4, 2, and 1.

1. First, multiply them together. Then, add them together.
2. Finally, subtract the addition result from the multiplication result.

	n = 4421		
4	4	2	1
digit1	digit2	digit3	digit4
Multiply the digits		4 x 4 x 2 x 1 = 32	
Add the digits		4 + 4 + 2 + 1 = 11	
FINAL RESULT		32 - 11 = 21	

Step 3. Identify the pattern

The main pattern is repeated digit extraction using arithmetic operations.

Java provides two operators that make this problem straightforward.

A. Modulo operator %

The expression n % 10 returns the last digit of a positive integer.

For example:

- 234 % 10 = 4
- 23 % 10 = 3
- 2 % 10 = 2

B. Integer division /

The expression n / 10 removes the last digit when both operands are integers and n is non-negative.

For example:

- 234 / 10 = 23
- 23 / 10 = 2
- 2 / 10 = 0

Together, these operations allow us to process the digits from right to left without converting the number into a stri ng.

How digit extraction works:-

234 4

$\xrightarrow{\% 10}$

234 23

$\xrightarrow{/ 10}$

Extract the last digit, process it, and remove it. Repeat until no digits remain.

We also need two accumulators:

- product, initialized to 1, because multiplying by 1 preserves the first digit.
- sum, initialized to 0, because adding 0 does not change the first digit.

We repeat this process in a while loop until n becomes zero.

Step 4. Build the logic

Algorithm :-

1. Initialize product = 1 and sum = 0.
2. While n > 0:
    i. Extract the last digit using n % 10.
    ii. Multiply that digit into product.
    iii. Add that digit to sum.
    iv. Remove the last digit using n / 10.
3. Return product - sum.

Pseudocode :-

```
FUNCTION subtractProductAndSum(n):
   product = 1
   sum = 0

   WHILE n > 0:
      digit = n % 10
      product = product * digit
      sum = sum + digit
      n = n / 10

    RETURN product - sum
```

NOTE :-

Why initialize product to 1 instead of 0?

Because if we initialize it to zero, every multiplication would remain zero. Starting at one allows the digits to build the correct product.

Why update n using n / 10?

If we do not remove the last digit, the loop will keep processing the same number indefinitely.

Step 5. Code in Java

This is the standard arithmetic solution for LeetCode 1281.

```
class Solution {
   public int subtractProductAndSum(int n) {
      int product = 1;
      int sum = 0;

      while (n > 0) {
         int digit = n % 10;
         product = product * digit;
         sum = sum + digit;
         n = n / 10;
      }

      return product - sum;
   }
}
```

How to submit

1. Open LeetCode Problem 1281.
2. Select Java as the programming language.
3. Enter the method in the provided Solution class, replacing the existing method if necessary.
4. Click Run to test the examples.
5. Click Submit after the tests pass.

Step 6. Explanation & dry-run

Explanation of the code :-

Statement	Purpose
int product = 1;	Stores the product of digits.
int sum = 0;	Stores the sum of digits.
while (n > 0)	Continues while digits remain.
int digit = n % 10;	Extracts the last digit.
product = product * digit;	Updates the running product.
sum = sum + digit;	Updates the running sum.
n = n / 10;	Removes the last digit.
return product - sum;	Returns the required difference.

Dry run: num = 234

Let's trace n = 234 iteration by iteration. Initially:

- product = 1
- sum = 0
- n = 234

Iteration	n	Extracted digit	product	sum	Updated n
1	234	4	4	4	23
2	23	3	12	7	2
3	2	2	24	9	0

Once n = 0, the loop terminates.

The return statement calculates: 24 - 9 = 15

Output: 15.

7. Analyze complexity and optimization

Let d represent the number of digits in n.

Time complexity: (O(d))

Each loop iteration processes exactly one digit. Since there are (d) digits, the loop executes (d) times.

Equivalently, for a positive integer (n), the time complexity can be expressed as (O(logn)), because the number of de cimal digits grows logarithmically with the value of \(n\).

Auxiliary space complexity: (O(1))

We use a fixed number of integer variables regardless of the number of digits. No additional arrays, lists, or strings a re required.

NOTE:-

Can it be optimized?

This solution is already optimal for the normal digit-by-digit approach : every digit must contribute to the product and sum. Processing each digit once is sufficient, and extra space remains constant.

Converting the number into a string is another possible approach, but it requires string processing and does not im prove the asymptotic complexity.

SCREENSHOT:-

